Electronics Fundamentals for Robotics & Circuit Building

A Practical Reference

Table of Contents

1. Passive Components (Resistors, Capacitors, Inductors, Diodes)
2. Reading Component Values
3. Active Components (Transistors, MOSFETs)
4. Key ICs You'll Actually Use
5. Voltage Regulation
6. Filters
7. When to Use What — Decision Guide
8. Functional Example Circuits
9. Digital vs. Analog I/O
10. Microcontrollers: Arduino, ESP32, Raspberry Pi
11. Power Supply Fundamentals
12. Prototyping, Tools & Safety
13. Suggested Learning Path

---

1. Passive Components

Resistors

Limit current flow. Used constantly — current limiting (LEDs), pull-up/pull-down for digital pins, voltage dividers, timing (with capacitors).

Key spec beyond resistance value: power rating (watts). A standard through-hole resistor is usually **1/4W (0.25W)** — fine for signal-level circuits, but check the math for anything driving real current (see Ohm's law calculations in Section 7).

Capacitors

Store and release electrical charge. Fundamentally different uses depending on type:

Type	Characteristics	Typical Use
Ceramic	Small, cheap, no polarity, low capacitance (pF to a few µF)	Decoupling/filtering noise near ICs, high-frequency filtering
Electrolytic	Larger capacitance (µF to thousands of µF), **has polarity** — reversing it can cause it to fail/vent/explode	Power supply smoothing, bulk energy storage
Tantalum	Compact, stable, has polarity, more expensive	Precision analog circuits, space-constrained designs
Film	Very stable, no polarity, larger physical size for given capacitance	Audio circuits, timing circuits needing precision

Golden rule: put a small ceramic capacitor (0.1µF, often written "104") close to the power pins of *every* IC you use. This is called a **decoupling capacitor** — it suppresses tiny voltage spikes/noise that would otherwise cause erratic IC behavior. Skipping this is one of the most common causes of "my circuit works on the breadboard but glitches randomly."

Inductors

Resist changes in current (the opposite tendency of a capacitor, which resists changes in voltage). Less commonly hand-selected by hobbyists directly, but they're the key component inside every buck/boost converter module you've already been using (like your SY8205) — they store energy magnetically during switching to smooth the output.

Diodes

Allow current to flow in one direction only.

Type	Use
Standard rectifier diode (1N4001-1N4007 family)	AC-to-DC conversion, reverse-polarity protection
LED	Light-emitting — always needs a current-limiting resistor in series
Zener diode	Clamps voltage at a specific level — used for simple voltage reference/protection
Schottky diode	Faster switching, lower voltage drop — used in efficient power circuits, flyback protection for motors/relays

Flyback diode: whenever you switch a motor, relay, or any inductive load with a transistor, put a diode across the load (cathode to positive) to absorb the voltage spike generated when current suddenly stops — without this, you can destroy the switching transistor.

---

2. Reading Component Values

Resistor Color Code (4-band, most common)

Each band is a digit or multiplier, read left to right:

Color	Digit	Multiplier
Black	0	×1
Brown	1	×10
Red	2	×100
Orange	3	×1,000
Yellow	4	×10,000
Green	5	×100,000
Blue	6	×1,000,000
Violet	7	—
Grey	8	—
White	9	—
Gold	—	×0.1 (tolerance ±5%)
Silver	—	×0.01 (tolerance ±10%)

Example: Brown-Black-Red-Gold = 1, 0, ×100, ±5% → **1,000Ω (1kΩ), 5% tolerance**

The 4th band (gold/silver/brown) is the **tolerance** — how much the actual value can vary from nominal. For most hobby work, 5% is fine.

Capacitor Codes

Values printed directly (e.g., "10µF", "0.1µF") are self-explanatory. But small ceramic capacitors often use a **3-digit code**:

First two digits = value, third digit = multiplier (in picofarads, pF)

Example: **"104"** = 10 × 10^4 pF = 100,000 pF = **0.1µF** (this is the standard decoupling capacitor value mentioned above)

Example: **"475"** = 47 × 10^5 pF = 4,700,000 pF = **4.7µF**

SMD (Surface-Mount) Marking Basics

Surface-mount resistors often use a 3 or 4-digit code similar to capacitors (e.g., "103" = 10×10^3 = 10,000Ω = 10kΩ). Surface-mount capacitors are frequently **unmarked entirely** due to size — you'll need to check the schematic/datasheet or measure with an LCR meter if you're reverse-engineering a board.

---

3. Active Components

Transistors — The Basic Switch/Amplifier

A transistor is fundamentally a **controllable switch/valve** — a small signal at one terminal controls a much larger flow of current between the other two terminals. This single concept underlies almost all modern electronics, from a simple LED driver to the billions of transistors inside a CPU.

BJT (Bipolar Junction Transistor) **3 terminals**: Base, Collector, Emitter. **Controlled by current** — a small current into the Base allows a much larger current to flow from Collector to Emitter. The ratio between them is the **hFE (current gain)**, typically 100-300 for common types.

- **NPN** (e.g., 2N2222, BC547) — the more common type, switches on when Base is more positive than Emitter
- **PNP** (e.g., 2N2907) — mirror-image type, switches on when Base is more negative than Emitter

Example: switching an LED that needs more current than a microcontroller pin can safely supply directly (most microcontroller pins max out around 20-40mA):

```
Microcontroller pin ── Resistor (~1kΩ) ── Base
                                  Collector ── LED ── Resistor ── V+
                                  Emitter ── GND
```

The small Base current (via the resistor) allows a much larger Collector-to-Emitter current to flow — the transistor does the "heavy lifting" the microcontroller pin couldn't do alone.

MOSFET (Metal-Oxide-Semiconductor Field-Effect Transistor) **3 terminals**: Gate, Drain, Source. **Controlled by voltage, not current** — once the Gate reaches a threshold voltage relative to the Source, the MOSFET switches fully on and draws essentially **zero continuous current** to stay there (unlike a BJT, which needs continuous base current). This makes MOSFETs far more efficient for switching, especially in battery-powered projects.

- **N-channel** (e.g., IRLZ44N) — far more common in hobby projects
- **P-channel** — less common, used in specific high-side switching situations

Critical detail — logic-level vs. standard: many MOSFETs (like the classic IRFZ44N) need **10-12V** at the gate to fully switch on — a 3.3V/5V microcontroller pin can't drive them properly, leaving them partially on and wasting power as heat. **Logic-level MOSFETs** (like IRLZ44N — note the "L") are specifically designed to switch fully on at 3.3V-5V, making them the correct choice for direct microcontroller control.

BJT vs. MOSFET — When to Pick Which

	BJT	MOSFET (logic-level)
Control signal	Current	Voltage
Switching efficiency	Lower (continuous base current wastes some power)	Higher (near-zero holding current)
Good for	Small/simple loads, cheap and simple designs	Motors, higher-current loads, anything battery-efficiency-sensitive
Common hobby parts	2N2222, BC547	IRLZ44N (logic-level), IRFZ44N (needs a gate driver — do not drive directly from a microcontroller pin)

Practical rule: for anything switching real current in a battery-powered robot/project (motors, higher-power LEDs, relays) — use a logic-level MOSFET. For simple, low-current signal-level switching — a basic BJT is fine and cheaper.

Transistors as Amplifiers (Beyond Switching) Both BJTs and MOSFETs can also operate in their **linear/active region** (partially on, not fully on/off) to amplify a small signal into a larger one — this is how audio amplifiers, radio circuits, and analog sensor conditioning work. This is a genuinely different design discipline from switching (needs careful biasing and gain calculation). For robotics/microcontroller work, you'll use transistors almost exclusively in **switching mode** — worth knowing amplification mode exists, but not essential to master right away.

Why You Rarely Build Your Own Motor Driver

The H-bridge circuits (TB6612FNG, L298N) you've already used in your robot build are essentially **4 MOSFETs or BJTs arranged to allow current flow in either direction** through a motor, safely. Building this from individual transistors is a good learning exercise, but for real projects, buying the pre-built driver IC is faster, safer, and already handles edge cases (like preventing both high and low side switches from conducting simultaneously, which would short the power supply).

---

4. Key ICs You'll Actually Use

IC	Function	Typical Use Case
555 Timer	Configurable timing/oscillator chip — astable (continuous pulsing) or monostable (single timed pulse) modes	Blinking LEDs, simple PWM generation, basic tone generation, timing delays without a microcontroller
LM358 (Op-Amp)	Amplifies the difference between two input voltages	Sensor signal amplification, simple voltage comparators, audio preamps
LM386 (Audio Power Amp)	Low-power audio amplifier, single supply, ×20 default gain (×200 with a gain capacitor)	Driving small speakers directly from a weak audio signal — see Circuit 7 in Section 8
7805 / AMS1117	Linear voltage regulator (fixed output, e.g., 5V or 3.3V)	Simple, cheap regulation when efficiency doesn't matter and input-output voltage difference is small
74HC595 (Shift Register)	Controls multiple outputs (e.g., 8 LEDs) using just 3 microcontroller pins	Expanding limited GPIO pins, driving LED displays
ULN2003 (Darlington Array)	7 channels of high-current switching, commonly paired with small stepper motors	Driving small steppers, relays, or other moderate-current loads directly from logic-level signals
PCF8574 (I2C GPIO expander)	Adds 8 GPIO pins accessible over I2C (2 wires)	Exactly what your LCD I2C backpack used — expanding pins over a simple 2-wire bus
ADS1115 (I2C ADC)	Adds 4 channels of precise analog-to-digital conversion over I2C	Reading analog sensors on boards (like the Pi) that lack a built-in ADC

---

5. Voltage Regulation

Linear vs. Switching (Buck) Regulators

	Linear (e.g., 7805)	Switching/Buck (e.g., your SY8205)
Efficiency	Poor — wastes excess voltage as heat	High — typically 85-95%+
Complexity	Very simple, few parts	More complex internally, but usually sold as a ready module
Heat generation	Significant if input/output voltage gap is large	Minimal

	Linear (e.g., 7805)	Switching/Buck (e.g., your SY8205)
Noise	Very clean output	Slightly noisier (switching frequency ripple), usually not an issue
Best for	Small voltage drops, low current, simplicity matters most	Anything with meaningful current draw or a large voltage drop — like stepping your robot's 11.1V down to 5V

Rule of thumb: if you're dropping more than ~2-3V and drawing more than a few hundred mA, use a switching/buck regulator — a linear regulator would waste significant power as heat and could overheat. This is exactly why your robot build uses the SY8205 (buck) rather than a simple 7805 to go from 11.1V to 5V.

---

6. Filters

What a Filter Actually Does

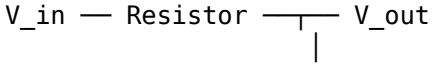
A filter shapes a signal based on **frequency** — passing some frequencies through while blocking or attenuating others. In robotics/electronics work, filters show up in two main contexts: **cleaning up power supplies** (removing ripple/noise from a DC rail) and **conditioning signals** (removing unwanted noise from a sensor reading, or shaping a control signal).

The Four Basic Filter Types (by frequency response)

Type	Passes	Blocks	Typical Use
Low-pass	Low frequencies	High frequencies	Removing high-frequency noise, smoothing a signal
High-pass	High frequencies	Low frequencies	Removing DC offset/slow drift, isolating fast changes
Band-pass	A specific frequency range	Everything outside that range	Radio tuning, isolating a specific sensor signal frequency
Band-stop (notch)	Everything except a specific range	One specific frequency	Removing a known noise frequency (e.g., 50/60Hz mains hum)

Passive RC Filters (Resistor + Capacitor) — the simplest, most common

Low-pass RC filter:



```
                Capacitor
                   │
                  GND
```

Cutoff frequency: `f_c = 1 / (2π × R × C)`

Frequencies below `f_c` pass through mostly unaffected; frequencies above `f_c` are increasingly attenuated. This exact circuit is extremely common for cleaning up a noisy analog sensor reading before it reaches an ADC pin.

High-pass RC filter — same two components, swapped positions:

```
V_in ── Capacitor ──┬── V_out
                    │
                 Resistor
                    │
                   GND
```

Worked example: with R = 10kΩ and C = 1µF:

`f_c = 1 / (2π × 10,000 × 0.000001) ≈ 15.9 Hz`

A low-pass filter with these values would smooth out fast electrical noise while passing slower signal changes (like a slowly-varying sensor reading) through cleanly.

LC Filters (Inductor + Capacitor) — used in power electronics

More effective than RC filters at power-supply frequencies, and don't dissipate as much energy as heat (inductors and capacitors both *store* energy rather than burning it off like a resistor does). This is exactly the combination inside every buck converter you've used (like your SY8205) — the inductor and output capacitor together smooth the rapidly-switching internal voltage into the clean, steady DC output you actually measure with a multimeter.

Power Supply Ripple Filtering

After rectifying AC to DC (or after a switching regulator's internal switching), the raw output isn't perfectly smooth — it has **ripple**, small unwanted fluctuations. **Bulk electrolytic capacitors** (typically 100-1000µF+) placed across the output absorb these fluctuations, and smaller ceramic capacitors (the same 0.1µF decoupling caps from Section 1) handle the faster, smaller residual noise that the bulk capacitor is too slow to catch. **Using both together — one bulk electrolytic, one small ceramic, in parallel — is standard practice**, not redundant.

"Current Filters" — Noise Suppression and Current-Related Protection

A few distinct things fall under this umbrella, worth distinguishing clearly:

Component/Technique	Purpose
Ferrite bead/choke	A small component threaded onto a power or signal wire that suppresses high-frequency noise/EMI without affecting the intended DC or low-frequency signal — common on USB cables and motor power leads to reduce electrical interference
Common-mode choke	Similar purpose, specifically targets noise that appears identically on both power and ground lines together

Component/Technique	Purpose
Decoupling capacitor (revisited)	As covered in Section 1 — technically a localized high-frequency filter right at an IC's power pins
Current-limiting resistor	Not a frequency filter — limits the *magnitude* of current (covered in Section 1/7), a different concept from frequency filtering despite the similar-sounding name
PTC resettable fuse	A current-limiting protection device that increases resistance sharply when current exceeds a threshold (heats up, self-limits), then resets automatically once the fault clears and it cools down — unlike a one-time blow fuse
NTC thermistor (inrush limiter)	Placed in series with power input, starts with higher resistance when cold (limiting the initial current surge when powering on), then drops to low resistance during normal operation

Practical relevance to your builds: the ripple you might see on your robot's motor power rail (from motor brush noise or switching) is exactly the kind of interference a ferrite bead on the motor's power leads helps suppress — worth adding if you ever notice your ESP32 or Pi behaving erratically specifically when the motors are running, since that's a classic symptom of motor electrical noise coupling into your logic circuits through a shared, insufficiently filtered power rail.

Active Filters (Op-Amp Based)

RC and LC filters are **passive** — no amplification, and the resistor/capacitor values interact with whatever's connected afterward (loading effects). **Active filters** add an op-amp to the circuit, giving more precise control over the cutoff frequency and the steepness of the transition, independent of what's connected downstream. This is a step up in complexity mainly relevant for precision audio or instrumentation work — for typical robotics sensor cleanup, a simple passive RC filter is usually sufficient.

7. When to Use What — Decision Guide

Situation	Use
Need to limit current to an LED	Resistor (calculate value — see Section 7)
Need to filter noise near an IC's power pins	0.1µF ceramic capacitor
Need to smooth a rectified DC power supply	Electrolytic capacitor (larger value, e.g., 100-1000µF)
Need to switch a motor/high-current load from a microcontroller pin	Logic-level MOSFET, or a pre-built driver IC (TB6612FNG, L298N)
Need to switch a small, low-current load (single LED, small relay coil)	BJT transistor is often sufficient and cheaper
Need a stable digital "high" or "low" when a pin isn't actively driven	Pull-up or pull-down resistor (typically 10kΩ)
Need to protect a transistor switching a motor/relay	Flyback diode across the load

Situation	Use
Need to drop voltage with minimal complexity, low current, small voltage gap	Linear regulator (7805, AMS1117)
Need to drop voltage efficiently with meaningful current or large voltage gap	Switching/buck regulator module
Need more digital output pins than your microcontroller has	Shift register (74HC595) or I2C GPIO expander (PCF8574)
Need to read an analog sensor but your board's ADC isn't precise/available enough	Dedicated ADC IC (ADS1115) over I2C

---

8. Functional Example Circuits

Circuit 1: LED Current-Limiting Resistor (the calculation everyone should know)

`Microcontroller Pin (e.g., 3.3V or 5V) ── Resistor ── LED (anode → cathode) ── GND`

Ohm's Law calculation:

```
R = (V_source − V_LED) / I_LED
```

For a standard red LED (V_LED ≈ 2V, I_LED ≈ 15mA) on a 5V pin:

```
R = (5V − 2V) / 0.015A = 200Ω
```

A 220Ω resistor (a standard, easily available value) is the typical practical choice — close enough, and errs slightly toward protecting the LED rather than exactly maximizing brightness.

Circuit 2: Voltage Divider (for sensor reading)

`V_in ── R1 ── [tap point → ADC pin] ── R2 ── GND`

Formula: `V_out = V_in × (R2 / (R1 + R2))`

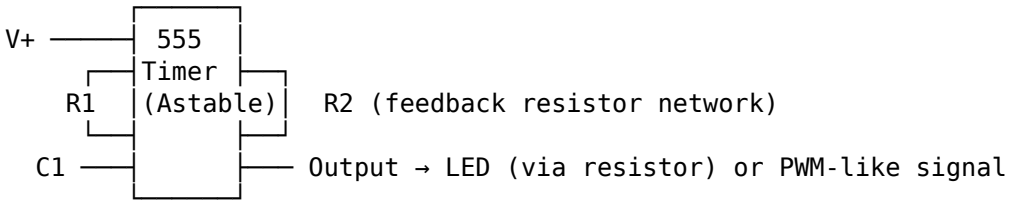
This is exactly the circuit you used for the HC-SR04 sonar's Echo line (stepping 5V down to a safe ~3.3V for an ESP32 GPIO) — the same fundamental circuit shows up everywhere: reading a higher voltage safely, or converting a variable-resistance sensor (like an LDR light sensor or a thermistor) into a readable voltage.

Circuit 3: Pull-Up Resistor for a Button Input

```
3.3V/5V ── Resistor (10kΩ) ── [tap point → digital input pin]
                                          |
                                       Button
                                          |
                                         GND
```

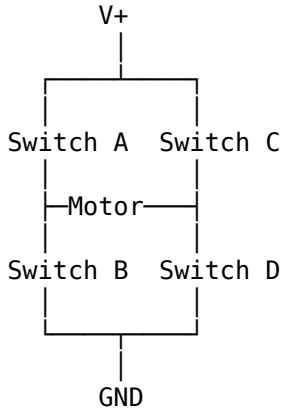
Without this resistor, an unpressed button leaves the input pin "floating" — reading random noise instead of a clean HIGH or LOW. The pull-up resistor ensures the pin reads a clean **HIGH** when the button is unpressed, and **LOW** when pressed (since pressing connects the pin directly to ground, overpowering the weaker pull-up). Most modern microcontrollers (ESP32, Arduino) have this resistor built in via software (`INPUT_PULLUP` mode) — genuinely useful to know both the internal option and how to build it discretely.

Circuit 4: Simple 555 Timer Astable (Blinking LED / Basic PWM)



The 555 Timer in astable mode continuously charges and discharges a capacitor (C1) through resistors (R1, R2), producing a repeating on/off output. **Frequency is set by the R1/R2/C1 values** (standard formula available in any 555 datasheet/calculator) — this is how you'd blink an LED or generate a basic tone/PWM signal **without any microcontroller at all**, useful for understanding timing circuits at the component level before relying on a microcontroller's built-in timer functions.

Circuit 5: Basic H-Bridge Concept (Why Motor Drivers Work the Way They Do)



Closing **A and D** (diagonal pair) drives the motor one direction. Closing **B and C** (the other diagonal pair) reverses it. **Never close A and B together, or C and D together** — that directly shorts V+ to GND. This is exactly the logic your TB6612FNG/L298N modules implement internally with transistors/MOSFETs instead of mechanical switches, with built-in protection against the "both switches on the same side" short-circuit scenario.

Circuit 6: Simple Op-Amp Comparator (Threshold Detector)

```
Sensor voltage (e.g., from an LDR voltage divider) ── Op-Amp (+) input
Reference voltage (set via a potentiometer)         ── Op-Amp (−) input
Op-Amp output ── goes HIGH or LOW depending on which input is greater
```

This is how a simple "trigger when light level drops below X" or "trigger when temperature exceeds Y" circuit works **without needing a microcontroller and code at all** — genuinely useful for simple, reliable threshold-triggered behavior (e.g., an automatic night light), and a good way to understand what your microcontroller's ADC + `if` statement is actually replacing when you do it in software instead.

Circuit 7: LM386 Audio Amplifier (Driving a Small Speaker)

The **LM386** is the classic, extremely common low-power audio amplifier IC — cheap, simple, runs from a single supply (4-12V), and is the standard building block for exactly the kind of

small-speaker projects you've already been wiring (conceptually the same job your Waveshare sound card's onboard amp is doing internally, just as a discrete, build-it-yourself version).

Basic circuit (default gain ×20):

```
Audio signal in ── Capacitor (10µF) ── Pin 3 (+ input)
Pin 2 (– input) ── GND
Pin 4 ── GND
Pin 6 ── V+ (e.g., 9V, or your 1S/2S battery rail)
Pin 5 (output) ── Capacitor (250µF) ── Speaker (+)
Speaker (–) ── GND
Pin 7 ── Capacitor (0.1µF) ── GND   (bypass, reduces supply noise)
```

Gain-boost option (×200): bridge **Pin 1 to Pin 8** through a 10µF capacitor. Left disconnected, the LM386 runs at its default ×20 gain — the boosted ×200 mode is useful when your input signal (e.g., straight from a microcontroller's low-level audio output) is too weak to drive a speaker to a useful volume at the lower gain setting.

Volume control (optional): replace the fixed input coupling path with a **potentiometer** (10kΩ) between the audio source and Pin 3 — turning it adjusts how much signal reaches the amplifier's input, giving manual volume control.

Why the capacitors matter here specifically: - **Input capacitor (10µF)**: blocks any DC offset from the source signal, passing only the actual audio (AC) signal through — a real-world application of the high-pass filter concept from Section 6 - **Output capacitor (250µF)**: blocks DC from reaching the speaker coil directly (which would just waste power as heat and offer no sound), while passing the audio signal through - **Pin 7 bypass capacitor (0.1µF)**: same decoupling principle from Section 1 — keeps supply noise from leaking into the amplified audio as hum/hiss

Power supply note: this is a good candidate for the **bulk + small capacitor combination** from Section 6's power supply ripple filtering — add a larger electrolytic (e.g., 100µF) directly across V+/GND near the chip if you notice any hum, especially if powering from a switching buck converter rather than a clean linear supply.

Speaker impedance matching: LM386 is commonly used with **8Ω speakers**, though it can drive 4Ω loads at reduced maximum output — relevant directly to the speaker impedance-matching question from your own robot build (checking whether your JBL teardown driver was 4Ω or 8Ω applies exactly the same way here).

---

9. Digital vs. Analog I/O

Before looking at specific microcontrollers, it's worth understanding the fundamental distinction in how *any* microcontroller pin can behave — this governs what you can and can't do with a given pin, on every board you'll ever use.

Digital I/O — Only Two States

A digital pin reads or outputs only **HIGH** or **LOW** — nothing in between. In practice, a pin is read as HIGH above roughly 60-70% of the supply voltage (e.g., above ~2.3V on a 3.3V system) and LOW below roughly 30% — there's a real analog voltage underneath, but the microcontroller collapses it into a binary decision.

- **Digital input**: reading a button, a switch, or any sensor that itself outputs a clean HIGH/LOW signal (e.g., many simple motion sensors, limit switches)

- **Digital output**: turning an LED fully on/off, driving a relay, sending a HIGH/LOW signal to another chip

Analog Input — A Continuous Range, Read via ADC

Real-world signals (a potentiometer's position, a light sensor's brightness, a battery's exact voltage) vary continuously, not just between two states. An **ADC (Analog-to-Digital Converter)** — built into most microcontrollers — samples this continuous voltage and converts it into a number your code can use.

Resolution matters: a 10-bit ADC (common on Arduino) divides the input voltage range into 1,024 steps (0-1023). A 12-bit ADC (common on ESP32) divides it into 4,096 steps (0-4095) — finer resolution, more precise readings.

Worked example: on an Arduino (10-bit ADC, 5V reference), reading a raw value of 512 corresponds to:

```
V = (raw_value / 1023) × V_reference = (512 / 1023) × 5V ≈ 2.5V
```

Not every pin can do this — check your specific board's pinout; only pins explicitly marked as analog-capable (often labeled "A0, A1…" on Arduino, or specific ADC-capable GPIOs on ESP32) support analog input.

Analog Output — Mostly Faked via PWM

Genuinely continuous analog output requires a **DAC (Digital-to-Analog Converter)** — present on some boards (ESP32 has 2 DAC-capable pins) but absent on many others (standard Arduino Uno has none at all).

What most "analog output" actually is: PWM (Pulse Width Modulation). Instead of outputting a true intermediate voltage, the pin rapidly switches fully HIGH and fully LOW, and varies the **proportion of time spent HIGH** (the "duty cycle") to simulate an average voltage. A 50% duty cycle at 5V behaves, to a slow-responding load like an LED or DC motor, much like a steady 2.5V would — even though the pin itself is only ever truly at 0V or 5V.

- **LED brightness control**: PWM duty cycle directly controls perceived brightness
- **DC motor speed control**: exactly what your TB6612FNG's PWMA/PWMB inputs use — this is precisely why your ESP32 code uses `ledcWrite()` with a value from 0-255, setting the PWM duty cycle that determines motor speed
- **Servo control**: technically a specific, precise PWM pulse *width* (not duty cycle in the same sense) tells a servo what angle to move to — the same underlying technique, applied slightly differently

Quick Reference Table

I/O Type	What it does	Example use	Needs
Digital Input	Reads HIGH/LOW	Button, switch, simple digital sensor	Any GPIO pin
Digital Output	Sets HIGH/LOW	LED on/off, relay trigger	Any GPIO pin
Analog Input	Reads a continuous voltage as a number	Potentiometer, LDR light sensor, battery voltage monitoring	An ADC-capable pin

I/O Type	What it does	Example use	Needs
Analog Output (PWM)	Simulates a continuous output via fast switching	LED dimming, motor speed, servo angle	A PWM-capable pin (most GPIOs on ESP32; specific pins on Arduino)
True Analog Output (DAC)	Outputs a genuine, real intermediate voltage	Audio signal generation, precise analog control voltages	A DAC-capable pin (rare — check your specific board)

---

10. Microcontrollers: Arduino, ESP32, Raspberry Pi

Arduino (e.g., Uno)

- **What it is**: a simple microcontroller board — runs one program in an infinite loop, no operating system
- **Strengths**: extremely beginner-friendly, huge community/tutorials, predictable real-time behavior (good for precise timing-critical tasks)
- **Limitations**: 8-bit, slow clock speed (16MHz), very little RAM (2KB), no built-in WiFi/Bluetooth
- **Use when**: you need simple, reliable, real-time control of motors/sensors/LEDs and don't need wireless connectivity or heavy computation

ESP32

- **What it is**: a more powerful microcontroller (32-bit, dual-core, 240MHz) with built-in WiFi and Bluetooth
- **Strengths**: fast enough for more complex logic, wireless built in, more GPIO pins, hardware PWM channels, built-in ADC/DAC
- **Limitations**: still a microcontroller — no operating system, no filesystem for complex software, can't run Python/vision libraries the way a full computer can
- **Use when**: you need the reliability/real-time behavior of a microcontroller, but also want WiFi/Bluetooth connectivity, or need more processing headroom than Arduino provides — this is exactly why your robot build uses ESP32 for motor control rather than Arduino

Raspberry Pi

- **What it is**: a genuine, full computer — runs a real operating system (Linux), supports a filesystem, multitasking, and full programming language ecosystems (Python, etc.)
- **Strengths**: can run computer vision (OpenCV, MediaPipe), machine learning models, complex decision-making software, connect to displays/cameras/USB peripherals like a normal PC
- **Limitations**: not truly "real-time" (the OS can introduce unpredictable small delays), boots slower, more power-hungry, more expensive
- **Use when**: you need actual "thinking" — vision processing, running trained models, complex logic, network services — exactly why your robot build uses the Pi for camera/gesture/voice, while the ESP32 handles the immediate motor control

Quick Comparison

	Arduino	ESP32	Raspberry Pi
Type	Microcontroller	Microcontroller	Full computer
OS	None	None (or RTOS)	Linux
Clock speed	16MHz	240MHz (dual-core)	1.5-2.4GHz+ (multi-core)
WiFi/Bluetooth	No	Yes, built-in	Yes, built-in
Real-time reliability	Excellent	Very good	Weaker (OS-dependent)
Can run Python/vision libraries	No	No (limited MicroPython only)	Yes, fully
Typical role in a robot	Simple, dedicated tasks	Motor/sensor control + wireless	Vision, decision-making, networking

---

11. Power Supply Fundamentals

Common Voltage Rails and Why They Exist

- **3.3V**: standard logic level for most modern microcontrollers/sensors (ESP32, most modern ICs)
- **5V**: standard for many older ICs, most servos, USB devices, Arduino Uno logic
- **12V**: common for motors, fans, some LED strips, and as the “raw” rail from many battery packs before being stepped down

Grounding — the Rule That Prevents 90% of Mystery Bugs

Every part of your circuit that shares signals must share a common ground reference. If your battery, motor driver, and microcontroller each have separate, unconnected ground paths, signals won’t read correctly (or at all) — this is the single most common cause of “my wiring looks right but nothing works” in beginner circuits, and it’s the rule we’ve repeated throughout every wiring diagram in your robot and plane builds.

Battery Chemistry Quick Reference

Type	Nominal Voltage/Cell	Notes
Li-ion (18650, etc.)	3.7V	High energy density, needs a proper charger and BMS/protection circuit
LiPo	3.7V	Similar to Li-ion but flexible pouch form factor, needs balance charging on multi-cell packs
NiMH	1.2V	Lower energy density, more tolerant of abuse, simpler charging
Alkaline (non-rechargeable)	1.5V	Simple, disposable, low power density

---

12. Prototyping, Tools & Safety

Essential Tools

- **Multimeter**: measures voltage, current, resistance, continuity — the single most important diagnostic tool, used throughout this whole course for confirming buck converter outputs, checking speaker impedance, and verifying wiring before powering on
- **Breadboard**: solderless prototyping — push components/wires into rows of connected holes, ideal for testing a circuit before committing to solder
- **Soldering iron**: for permanent connections — necessary once a breadboard prototype is proven and you want a reliable, vibration-resistant final build
- **Wire strippers/cutters**: for preparing wire ends cleanly

Breadboard Basics

The horizontal rows in the middle section are electrically connected in short strips (usually 5 holes); the outer rows (marked + and −) run the full length of the board and are meant for power/ground distribution. Components straddle the central gap to connect across the two half-sections.

Common Beginner Mistakes (Worth Knowing Upfront)

1. **Reverse polarity** on electrolytic capacitors or LEDs — check orientation before powering on
2. **Missing common ground** between separately-powered sections of a circuit
3. **Skipping current-limiting resistors** on LEDs — connecting an LED directly to a supply voltage without a resistor typically destroys it almost instantly
4. **Using a standard (non-logic-level) MOSFET** and expecting it to switch on fully from a 3.3V/5V microcontroller pin
5. **Forgetting flyback diodes** when switching motors/relays with a transistor
6. **Not checking voltage with a multimeter before connecting a sensitive component** — this single habit prevents the majority of "I fried my board" incidents

---

13. Suggested Learning Path

1. **Learn to read resistor color codes and capacitor markings** until it's second nature — you'll be doing this constantly
2. **Build Circuit 1 (LED + resistor) on a breadboard** — the simplest possible complete circuit, confirms your basic tools and understanding work
3. **Build Circuit 3 (pull-up + button)** — introduces reading a digital input properly
4. **Build Circuit 2 (voltage divider) with a light-dependent resistor (LDR)**, read the result with a microcontroller's ADC — connects the analog world to digital code
5. **Build Circuit 4 (555 timer astable)** — understand timing/oscillation without a microcontroller, appreciate what your code's timing functions are actually replacing
6. **Move to a microcontroller** (start with Arduino for simplicity, or ESP32 if you know you'll want wireless) — reimplement Circuits 1 and 3 in code instead of pure hardware logic, compare the experience
7. **Build a simple motor control circuit** — either from individual MOSFETs (Circuit 5 concept) to understand the H-bridge principle, or with a pre-built driver module (TB6612FNG) to see the same principle packaged for real use — exactly the progression used in your own robot build

8. **Add a Raspberry Pi** once you have a working microcontroller-based project and want to add vision, voice, or complex decision-making on top — the same two-tier architecture used throughout your Sahayak robot and any future autonomous plane work

This mirrors the same principle from the aeromodeling course: **understand the fundamental building block before reaching for the pre-packaged solution** — knowing why a decoupling capacitor, a flyback diode, or a pull-up resistor is there makes debugging someone else's (or your own) circuit vastly easier than treating every component as a magic black box.